

Labo 05 — Multi-stage et images de qualité production

Théorie — comment passer d'une image de 800 Mo qui contient votre code source à une image de 200 Mo qui ne contient que ce qui tourne.

Objectifs

- Comprendre pourquoi une image qui sert à **construire** ne doit pas être celle qui **exécute**.
 - Écrire un build **multi-stage** pour Spring Boot et pour Angular.
 - Choisir une image de base en connaissance de cause (Debian, Alpine, distroless).
 - Savoir ce que BuildKit apporte : cache, parallélisme, secrets.
 - Relier taille d'image, surface d'attaque et temps de déploiement.
-

1. Le problème : l'outillage de build reste dans l'image

Un premier Dockerfile naïf pour une API Java :

```
FROM maven:3.9-eclipse-temurin-21
WORKDIR /app
COPY . .
RUN mvn package -DskipTests
ENTRYPOINT ["java", "-jar", "/app/target/api.jar"]
```

Il fonctionne. Il produit une image de **800 Mo à 1 Go** qui contient : un JDK complet (compilateur, outils de débogage), Maven, le dépôt local `~/.m2` avec des centaines de JAR, votre **code source**, les tests, et le JAR final. En production, il ne sert que le JRE et le JAR : environ 250 Mo.

Les conséquences ne sont pas seulement esthétiques :

- **Sécurité.** Le code source part chez tous ceux qui ont l'image. Chaque outil embarqué (compilateur, `curl`, `git`, `shell`) est un moyen d'action supplémentaire pour un attaquant qui obtiendrait l'exécution de code, et une ligne de plus dans le rapport de scan de vulnérabilités.
- **Coût.** 800 Mo transférés à chaque déploiement, sur chaque nœud, stockés dans chaque registry, avec la rétention des versions antérieures.
- **Temps.** Le démarrage d'un conteneur inclut le téléchargement de l'image si elle est absente. Sur un incident de production à 3 h du matin, la différence se voit.

2. Le multi-stage

Un Dockerfile peut contenir **plusieurs** `FROM`. Chacun ouvre un *stage* — un environnement de construction indépendant. Seul le **dernier** produit l'image finale ; les autres sont jetés. Et `COPY --from=<stage>` permet de récupérer des fichiers dans un stage précédent.

```
# ----- stage 1 : construction -----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

# ----- stage 2 : exécution -----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/api.jar app.jar
USER 1000:1000
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

L'image finale contient **un JRE et un JAR**. Ni Maven, ni JDK, ni sources, ni tests, ni `.m2`. Rien de ce qui s'est passé dans le stage `build` n'y laisse de trace — pas même dans les couches masquées, puisque ces couches ne font tout simplement pas partie de l'image.

À retenir — Le multi-stage est aussi la seule protection réellement fiable contre les secrets de build : ce qui est copié dans un stage jeté n'existe pas dans l'image finale. Attention toutefois : `COPY --from=build /app /app` recopierait tout, secrets compris. On ne copie que l'artefact.

Le même schéma pour Angular :

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build # produit dist/

FROM nginx:alpine
COPY --from=build /app/dist/mon-app/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Point crucial : **Node ne survit pas** au build. Un front Angular en production n'est que du HTML, du CSS et du JavaScript statiques ; il n'a pas besoin d'un moteur JavaScript côté serveur. On ne conteneurise **jamais** `ng serve`, qui est un serveur de développement : lent, verbeux, sans compression, sans cache, et explicitement non prévu pour la production.

3. Choisir son image de base

Base	Taille	Avantages	Inconvénients
<code>debian</code> / <code>ubuntu</code>	75-120 Mo	Tout fonctionne, outillage complet, <code>glibc</code>	Lourde, beaucoup de paquets donc de CVE
<code>*-slim</code>	30-80 Mo	Bon compromis, reste du Debian	Moins d'outils installés
<code>alpine</code>	5-10 Mo	Très légère, <code>apk</code> efficace	Utilise musl et non <code>glibc</code>
<code>distroless</code>	20-50 Mo	Aucun shell, aucun gestionnaire de paquets	Débogage difficile, pas de <code>docker exec sh</code>

Le piège d'Alpine mérite un développement. Alpine remplace la bibliothèque C standard `glibc` par `musl`. La plupart des programmes s'en accommodent, mais pas tous : les binaires natifs compilés pour `glibc` refusent de démarrer, certaines bibliothèques Java natives (compression, cryptographie, pilotes) échouent, et des différences de résolution DNS ou de gestion des locales apparaissent. Des ralentissements ont aussi été mesurés sur l'allocation mémoire de certaines charges Python et Java.

En pratique, pour Spring Boot : `eclipse-temurin:21-jre-alpine` convient dans l'immense majorité des cas et divise la taille par deux ; en cas de dépendance native, on revient à `eclipse-temurin:21-jre-jammy`. Le choix se **teste**, il ne se décrète pas.

Les images `distroless` (Google) ne contiennent que le runtime et votre application : pas de shell, pas de `ls`, pas de gestionnaire de paquets. La surface d'attaque est minimale, mais `docker exec -it conteneur sh` ne fonctionne plus — il faut avoir prévu son observabilité autrement.

4. Ce qui pèse vraiment

Quatre leviers, par ordre d'efficacité décroissante :

1. **Le multi-stage** — supprime l'outillage de build. C'est le levier majeur : 800 Mo → 250 Mo.
2. **L'image de base** — `-jre` au lieu de `-jdk`, `alpine` au lieu de `debian`.
3. **Le `.dockerignore`** — évite d'embarquer `.git`, `node_modules`, `target`.
4. **Le regroupement installation/nettoyage** dans le même `RUN`.

En revanche, ce qui n'a **aucun** effet : supprimer des fichiers dans une couche ultérieure. Ils restent dans l'image (labo 02). Et le nombre de couches, à lui seul, ne change presque rien à la taille.

```
docker images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'
docker history mon-api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head
```

5. BuildKit

BuildKit est le moteur de construction par défaut depuis Docker 23. Il apporte quatre choses utiles :

- **Le parallélisme.** Les stages indépendants sont construits en même temps. Un Dockerfile qui construit le front et le back en deux stages les traite simultanément.
- **La construction paresseuse.** Un stage dont aucun fichier n'est copié dans l'image finale n'est pas construit du tout.

- **Les caches persistants.** `RUN --mount=type=cache,target=/root/.m2 mvn package` conserve le dépôt Maven **entre les builds**, sans l'inclure dans l'image. Sur un agent de CI, le gain est spectaculaire.
- **Les secrets.** `RUN --mount=type=secret,id=npnrc ...` rend un fichier disponible pendant une seule instruction, sans jamais l'écrire dans une couche (labo 08).

```
# syntax=docker/dockerfile:1
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN --mount=type=cache,target=/root/.m2 mvn -q package -DskipTests
```

La ligne `# syntax=docker/dockerfile:1` en tête active la dernière version de la syntaxe : elle est nécessaire pour ces montages.

6. Spring Boot : les couches du JAR

Un JAR Spring Boot pèse 50 Mo, dont 45 Mo de dépendances qui ne changent presque jamais et 5 Mo de code qui change à chaque commit. Copié en bloc, il forme une couche unique de 50 Mo retransférée intégralement à chaque déploiement.

Spring Boot sait se découper (`layertools`) :

```
FROM eclipse-temurin:21-jre-alpine AS extract
WORKDIR /app
COPY target/api.jar api.jar
RUN java -Djarmode=tools -jar api.jar extract --layers --destination extracted

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=extract /app/extracted/dependencies/ ./
COPY --from=extract /app/extracted/spring-boot-loader/ ./
COPY --from=extract /app/extracted/snapshot-dependencies/ ./
COPY --from=extract /app/extracted/application/ ./
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Les dépendances forment une couche stable, le code applicatif une petite couche volatile : le déploiement ne transfère plus que quelques Mo. Vous n'avez pas à retenir ce Dockerfile ; retenez le **principe**, qui est celui du labo 04 appliqué au contenu d'un JAR.

7. En entreprise

- **Un seul Dockerfile par service**, multi-stage, versionné avec le code. La CI n'a besoin ni de Maven ni de Node installés : `docker build` suffit, ce qui garantit que le build de la CI et celui du poste développeur sont identiques.
- **Les tests** tournent souvent dans un stage dédié (`RUN mvn test`), de sorte qu'un test rouge fasse échouer le build de l'image.
- **Le scan de vulnérabilités** (Trivy, Gype) s'applique à l'image finale. Une image minimale produit un rapport court, donc réellement traité — une image de 1 Go produit 300 CVE que

personne ne lira.

- **L'image finale tourne en non-root**, sur un port > 1024, sans shell si possible.
-

À retenir

- Ce qui construit ne doit pas exécuter : c'est tout l'objet du multi-stage.
- Plusieurs `FROM` = plusieurs stages ; seul le dernier devient l'image, `COPY --from` y récupère l'artefact.
- Node n'a rien à faire dans l'image finale d'un front Angular : on sert du statique avec nginx.
- `-jre` plutôt que `-jdk`, `alpine` si les dépendances natives le permettent, `distroless` si l'on accepte de perdre le shell.
- Alpine utilise `musl` et non `glibc` : à valider par un test, jamais par principe.
- BuildKit apporte parallélisme, caches persistants et secrets de build.
- Supprimer un fichier dans une couche ultérieure ne réduit pas la taille de l'image.

Vocabulaire

stage : étape de build ouverte par un `FROM`. — `COPY --from` : récupération de fichiers depuis un autre stage ou une autre image. — **image de base** : image citée par `FROM`. — **distroless** : image sans shell ni gestionnaire de paquets. — **musl / glibc** : deux implémentations de la bibliothèque C standard. — **BuildKit** : moteur de construction de Docker. — **cache mount** : cache persistant entre builds, hors image. — **surface d'attaque** : ensemble des composants exploitables présents dans l'image.